Sequential Task:
Swing Window Appearance

Object-Oriented Programming and GUI

May 14, 2026

1. The class `LAFFrame.java` is attached to this task. Place it in the project.
2. Go to the `TODO 01` marker and implement the code fragment responsible for setting the application appearance and launching the graphical interface in the appropriate thread.
   (a) Try to set the system look and feel of the application using `UIManager.setLookAndFeel(...)`.
      - Retrieve the system look and feel class name using `UIManager.getSystemLookAndFeelClassName()`.
      - Pass this value to the `setLookAndFeel(...)` method.
   (b) Place the look and feel setup call inside a `try-catch` block.
      - Catch the general `Exception`.
      - Ignore the exception if setting the look and feel fails.
   (c) Call the `SwingUtilities.invokeLater(...)` method to run GUI code in the Swing event dispatch thread.
   (d) Pass a lambda expression as the argument to the `invokeLater(...)` method.
      - Create a new instance of the `LAFFrame` class.
      - Call the `setVisible(true)` method to display the application window.
3. Go to the `TODO 02` marker and declare a class field responsible for storing a mapping of names to `String` values.
   (a) Create a private field `lafMap` of type `Map<String, String>`.
   (b) Initialize the field as a new instance of `HashMap`.
   (c) Ensure that the structure stores key-value pairs where both elements are of type `String`.
4. Go to the `TODO 03` marker and implement the code fragment responsible for retrieving available look and feel styles and preparing their textual representation and mapping.
   (a) Call the method `UIManager.getInstalledLookAndFeels()` and assign the result to the variable `lafs` of type `UIManager.LookAndFeelInfo[]`.
   (b) Create an array `lafNames` of type `String[]` with a length equal to `lafs.length`.

(c) Iterate through all elements of the `lafs` array using a `for` loop.

i. Retrieve the current style name by calling `lafs[i].getName()`.

ii. Assign this name to the array element `lafNames[i]`.

iii. Retrieve the style class name by calling `lafs[i].getClassName()`.

iv. Add an entry to the `lafMap`, where:

- the key is the style name `lafs[i].getName()`,
- the value is the style class name `lafs[i].getClassName()`.

5. Go to the `TODO 04` marker and implement the private method `applyLookAndFeel(String className)`, whose task is to change the current appearance of the application.

(a) Accept the parameter `className`, which represents the fully qualified class name of the selected look and feel style.

(b) Place the style switching logic inside a `try-catch` block.

(c) In the `try` block call the method `UIManager.setLookAndFeel(className)`.

- Set the selected style as the active application look and feel.

(d) Call the method `SwingUtilities.updateComponentTreeUI(this)`.

- Refresh the appearance of all components in the current window.

(e) In the `catch` block catch the `Exception` type exception.

(f) Display an error message using `JOptionPane.showMessageDialog(...)`.

- Pass the current component as the dialog owner.
- Append the exception message using `ex.getMessage()`.

6. Go to the `TODO 05` marker and implement the code fragment handling the selection of a look and feel style from the interface component and applying it.

(a) Retrieve the currently selected style name from the `lafComboBox` component.

- Call the `getSelectedItem()` method.
- Cast the result to the `String` type and assign it to the variable `selectedName`.

(b) Based on the selected name retrieve the corresponding style class name from the `lafMap`.

- Call the method `lafMap.get(selectedName)`.
- Assign the result to the variable `className`.

(c) Call the method `applyLookAndFeel(className)`.

- Apply the selected look and feel style in the application.

7. Run the application and check how selecting different interface styles from the drop-down menu works.

8. Go to the `TODO 06` marker and implement the private method `showJumpingDialog()`, whose task is to display a dialog window at a random position on the screen and handle user interaction.

   (a) Increase the displayed dialog counter by incrementing the `dialogCount` field.

   (b) Retrieve the screen size using `Toolkit.getDefaultToolkit().getScreenSize()` and store it in the variable `screenSize`.

   (c) Calculate the maximum dialog position coordinates.
   - Determine `maxX` as the screen width reduced by the constant dialog width.
   - Determine `maxY` as the screen height reduced by the constant dialog height.

   (d) Generate random coordinates `x` and `y`.
   - Use `random.nextInt(Math.max(1, maxX))` for the X axis.
   - Use `random.nextInt(Math.max(1, maxY))` for the Y axis.

   (e) Build the dialog message content using `String.format(...)`.
   - Include the dialog number `dialogCount`.
   - Include the coordinates `(x, y)`.
   - Retrieve the current style name using `UIManager.getLookAndFeel().getName()`.

   (f) Define the `options` array containing the available dialog buttons.

   (g) Create a `JOptionPane` object passing:
   - the prepared message,
   - the message type `JOptionPane.INFORMATION_MESSAGE`,
   - the option type `JOptionPane.DEFAULT_OPTION`,
   - no icon,
   - the options array as buttons,
   - the default option as the first element of the array.

   (h) Create a `JDialog` object by calling `optionPane.createDialog(this, ...)`.
   - Set the dialog title containing the `dialogCount` number.

   (i) Set the dialog position using `dialog.setLocation(x, y)`.

   (j) Display the dialog by calling `dialog.setVisible(true)`.

   (k) Retrieve the selected option using `optionPane.getValue()` and store it in the variable `selectedValue`.

   (l) Check whether the user selected the option causing another dialog to be displayed.
   - Compare the value of `selectedValue` with the appropriate string.
   - If the condition is met call `SwingUtilities.invokeLater(...)`.
   - In the provided lambda expression call the method `showJumpingDialog()` again.

9. Go to the `TODO 07` marker and complete the lambda with a call to the `showJumpingDialog()` method.

10. Test the application.

11. ★ Milestone 1: present the result of your work to the instructor